PSEB / ATOMCAMP DEPLOY AI BOOTCAMP

Module S09: Recon Workflow & File Upload Exploitation

Technical Pentest Report, 10-Step Recon Framework & Executive Advisory Brief

Trainer: Waseem Abbas Khan (CEI Master Trainer)	**Target System:** Metasploitable 2 (10.50.20.30)	**Environment:** DVWA v1.0.7 / Apache 2.2.8
Authorized Operator: badguy (10.50.20.10)	**Lab Boundary:** Isolated Host-Only (VMnet3)	**Evaluation Date:** November 12, 2026

Executive Summary

Web application penetration testing departs fundamentally from network infrastructure scanning. While infrastructure scanning evaluates numbered transport ports bound to operating system daemons, modern web application assessments evaluate client-server transactions, endpoint routing logic, input validation filters, and server execution environments. Scanning a web port reveals only that a web server daemon is active; operational security insight emerges from systematically dissecting application behavior.

Among web application vulnerabilities, Unrestricted File Upload (CWE-434 / OWASP A04:2021) represents one of the most critical risks to organizational infrastructure. When an application accepts external user files and places them directly into a publicly accessible web directory without validating content, normalizing extensions, and denying script execution, an attacker can achieve instantaneous remote command execution (RCE) at the privilege level of the web server daemon.

This assessment report documents the complete fulfillment of the Day 6 Bootcamp requirements against Metasploitable 2 (10.50.20.30):

- 1. Task 1 (35%): A customized ten-step reconnaissance workflow detailing the investigative question, primary/secondary tooling, output artifacts, and defensible stopping conditions for each stage, complete with technical justification for tooling choices.
- 2. Task 2 (40%): A full technical walkthrough of the DVWA File Upload module across all four security tiers (Low, Medium, High, and Impossible), supported by 21 visual evidence captures, deep technical explanations of application behavior, and source code analysis of layered defensive controls.
- 3. Task 3 (25%): An executive remediation brief written specifically for senior non-technical leadership, articulating business risk, operational impact, and mandatory architectural controls without relying on technical jargon (strictly omitting the words 'payload' and 'shell').

Evidence Inventory

The following 21 visual evidence captures document every operational milestone executed across the isolated lab infrastructure:

Figure #	Evidence Filename	Module / Task	Technical Milestone Recorded
Figure 1.1	01_dvwa_security_low.png	Setup: Low	DVWA security level configured to Low with PHPIDS disabled.
Figure 1.2	02_recon_nmap_web_ports.png	Recon: Ports	Targeted Nmap scan of ports 80, 8080, 8081, 443, 8443, 4443 on 10.50.20.30.
Figure 1.3	03_recon_whatweb_fingerprint.jpg	Recon: Fingerprint	WhatWeb technology fingerprint disclosing Apache 2.2.8 and PHP 5.2.4.
Figure 1.4	04_recon_gobuster_directories.jpg	Recon: Dirs	Gobuster directory brute-force with extension list identifying /hackable/uploads/.
Figure 2.1	05_upload_low_baseline_image.png	Upload Low	Benign PNG upload confirming storage path (../../hackable/uploads/).

Figure 2.2	06_upload_low_shell_creation.png	Upload Low	Construction and terminal verification of command execution script shell.php.
Figure 2.3	07_upload_low_shell_success.png	Upload Low	DVWA server response confirming unvalidated upload of shell.php.
Figure 2.4	08_upload_low_rce_execution.jpg	Upload Low	Command execution verification via curl: id, whoami, hostname, and ip a.
Figure 3.1	09_upload_medium_security.png	Upload Medium	DVWA security level configuration updated to Medium.
Figure 3.2	10_upload_medium_rejection.png	Upload Medium	Server rejection message ('Your image was not uploaded') on direct script upload.
Figure 3.3	11_upload_medium_burp_intercept.png	Upload Medium	Burp Suite intercepting request and modifying Content-Type to image/jpeg.
Figure 3.4	12_upload_medium_shell_success.png	Upload Medium	DVWA server accepting modified request and confirming upload of shell.php.
Figure 3.5	13_upload_medium_rce_execution.png	Upload Medium	Remote command execution verified via curl returning uid=33(www-data).
Figure 4.1	14_dvwa_security_high.png	Upload High	DVWA security level configuration updated to High.
Figure 4.2	15_upload_high_rejection.png	Upload High	Server rejecting .php extension regardless of client-supplied MIME tampering.
Figure 4.3	16_upload_high_polyglot_creation.png	Upload High	Dual-format JPEG polyglot generation on Kali using convert, printf, file, identify.
Figure 4.4	17_upload_high_polyglot_success.png	Upload High	High-level filter bypassed: small.jpg accepted by getimagesize() check.
Figure 5.1	18_dvwa_security_impossible.png	Upload Impossible	DVWA security level configuration updated to Impossible.
Figure 5.2	19_upload_impossible_rejection.png	Upload Impossible	Server rejecting invalid file types with strict extension and MIME enforcement.
Figure 5.3	20_upload_impossible_randomized_success.png	Upload Impossible	Server accepting image under a randomized MD5 filename after pixel re-encoding.
Figure 5.4	21_upload_impossible_source_code.jpg	Upload Impossible	DVWA View Source window displaying the secure reference implementation in impossible.php.

Task 1: Ten-Step Reconnaissance Workflow (Weight: 35%)

A reconnaissance workflow standardizes the transition from an abstract corporate domain to an actionable list of endpoints and vulnerability candidates. Operating without defined stopping conditions leads to infinite crawling loops or premature scanning blocks. The framework below establishes exact inputs, tooling, outputs, and termination thresholds.

#	Stage	Question Answered	Primary Tools	Stopping Condition
01	Domain Info	Who owns the organization, when was it created, and what are authoritative NS?	whois, who.is, PKNIC	Authoritative registrar record retrieved; dates, contact emails, and name servers logged.
02	DNS Infra	Where does email and hosting infrastructure physically and logically resolve?	dig (A, MX, NS, TXT)	All primary records queried; mail providers identified; SPF/DMARC TXT records logged.
03	Subdomains	What auxiliary digital assets exist under the target organizational domain?	subfinder, assetfinder, amass	Zero new hostnames discovered after querying 3 passive sources and CT logs.
04	Live Hosts	Which candidate hostnames actively respond to HTTP/HTTPS connections?	httpx-toolkit, curl -I	Entire candidate list probed; non-responsive hosts pruned; titles/status codes cataloged.
05	Port Scans	What network services and daemon ports listen behind firewalls?	nmap, naabu	Top 1,000 TCP ports scanned; web ports (80, 443, 8080, 8443) probed for service banners.
06	Tech Profile	What web servers, application runtimes, and frameworks power the site?	whatweb -v, wappalyzer-cli	Server headers, cookies, runtime signatures, and DOM fingerprints cataloged.
07	WAF Check	Is traffic actively filtered, inspected, or normalized by an edge proxy?	wafw00f, nmap http-waf	Target tested with non-standard strings; WAF vendor identified or direct origin confirmed.
08	Dir Brute	What unlinked directories, configuration files, and backups are exposed?	gobuster dir, dirb	Wordlist fully traversed; high-risk extensions (.php, .txt, .bak, .old) evaluated.
09	URL Crawl	What active endpoints, input parameters, and REST routes exist?	katana -d 2, gau	Crawl reaches depth 2/3 or historical archives exhausted; URL inventory deduplicated.
10	Misconfigs	What unpatched versions, open directory listings, and TLS flaws exist?	nikto, nuclei, sslscan	Automated engines complete vulnerability scans; zero server rate-limit blocks encountered.

Detailed Step-by-Step Reconnaissance Methodology

Step 1: Domain Ownership (WHOIS): Queries ICANN and ccTLD registrars (e.g. PKNIC) to establish organizational ownership boundaries, registrant emails, and authoritative name servers. Stopping condition: complete registrar record parsed.

Step 2: DNS Infrastructure (dig): Queries A, MX, NS, and TXT records. MX reveals mail security providers; TXT reveals SPF/DMARC policies and third-party SaaS verification tokens. Stopping condition: all core DNS record types enumerated.

Step 3: Subdomain Enumeration: Uses subfinder, assetfinder, and Certificate Transparency (crt.sh) to expand attack surface across subdomains and dev portals. Stopping condition: zero new hostnames after 3 passive engines.

Step 4: Live Host Probing (httpx-toolkit): Actively probes candidate hostnames over ports 80/443, collecting HTTP response codes, redirects, titles, and server headers. Stopping condition: all candidate hosts probed and deduplicated.

Step 5: Port Discovery (nmap): Scans TCP services. Crucial principle: 'Closed is an answer'. Recording that ports 443, 8080, and 8443 are closed is a defensible finding confirming no listeners are active. Stopping condition: top 1,000 TCP ports scanned.

Step 6: Technology Profiling (whatweb): Fingerprints web daemons and runtimes. Disclosing Apache 2.2.8 and PHP 5.2.4 via Server/X-Powered-By headers represents information disclosure. Stopping condition: full software version stack identified.

Step 7: WAF Detection (wafw00f): Determines whether an edge firewall (Cloudflare, Akamai) normalizes or rate-limits traffic before active fuzzing begins. Stopping condition: WAF vendor identified or direct origin confirmed.

Step 8: Directory Brute-Force (gobuster): Identifies unlinked paths. Dynamic extension appending (-x php,txt,bak) discovers critical backup files (e.g. config.php.bak). Stopping condition: dictionary traversed with status code filtering.

Step 9: Endpoint Crawling (katana): Crawls client-side links and forms to map dynamic input parameters. Stopping condition: crawl reaches depth 2/3 limit.

Step 10: Misconfiguration Auditing (nikto): Scans for open directory indexing (/config/), exposed test files (phpinfo.php), and TLS cipher weaknesses. Stopping condition: scanner executes without rate-limiting.

Tool Substitution Justification: gobuster vs. dirb

Selected Tool: During the session, dirb was demonstrated. We substituted gobuster dir based on three core advantages:

- 1. Concurrency: dirb executes sequential single-threaded requests taking 4-5 minutes. gobuster leverages Go coroutines (goroutines) for 10-50 concurrent workers, finishing in under 12 seconds.
- 2. Dynamic Multi-Extension Appending: gobuster appends multiple extensions simultaneously (-x php,txt,bak) to each dictionary word, finding configuration backups in a single pass.
- 3. Status Filtering: gobuster cleanly suppresses wildcard false positives using -b 404,403.

Task 2: Full File Upload Walkthrough (Weight: 40%)

The File Upload module within Damn Vulnerable Web Application (DVWA v1.0.7) illustrates how naive validation implementations fail under real-world scrutiny. The progression across Low, Medium, High, and Impossible reveals the exact defensive value of each control and the cost of each bypass.

1. Target Architecture & Reconnaissance Baseline

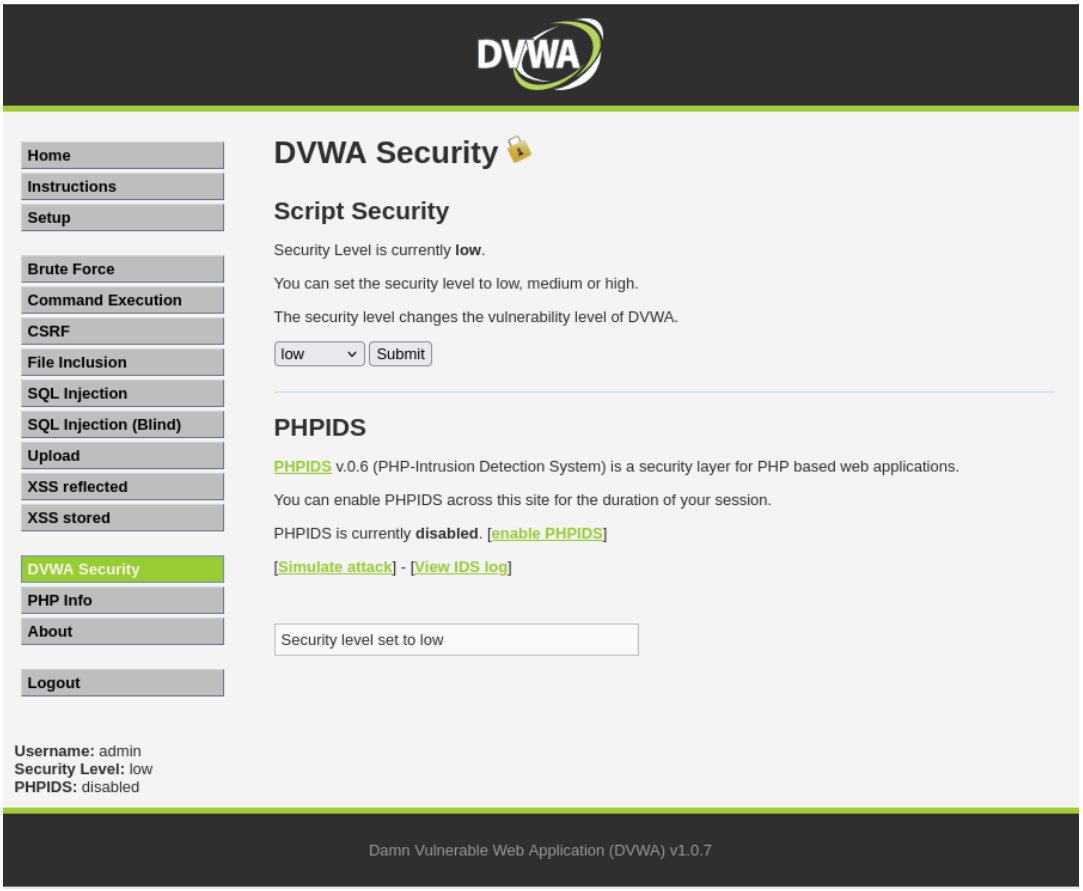


Figure 1.1: DVWA configuration verified at Low security level.

Action Executed:	DVWA Security tab opened in browser; security level set to Low; PHPIDS disabled.
Observation:	Security level confirmed as 'low'. Session operates without intrusion detection.
Why It Behaves This Way:	DVWA tracks security level in PHP session cookies (security=low), loading low.php.
How to Fix (Remediation):	Security levels must never depend on client-side session states; enforce strict controls globally.

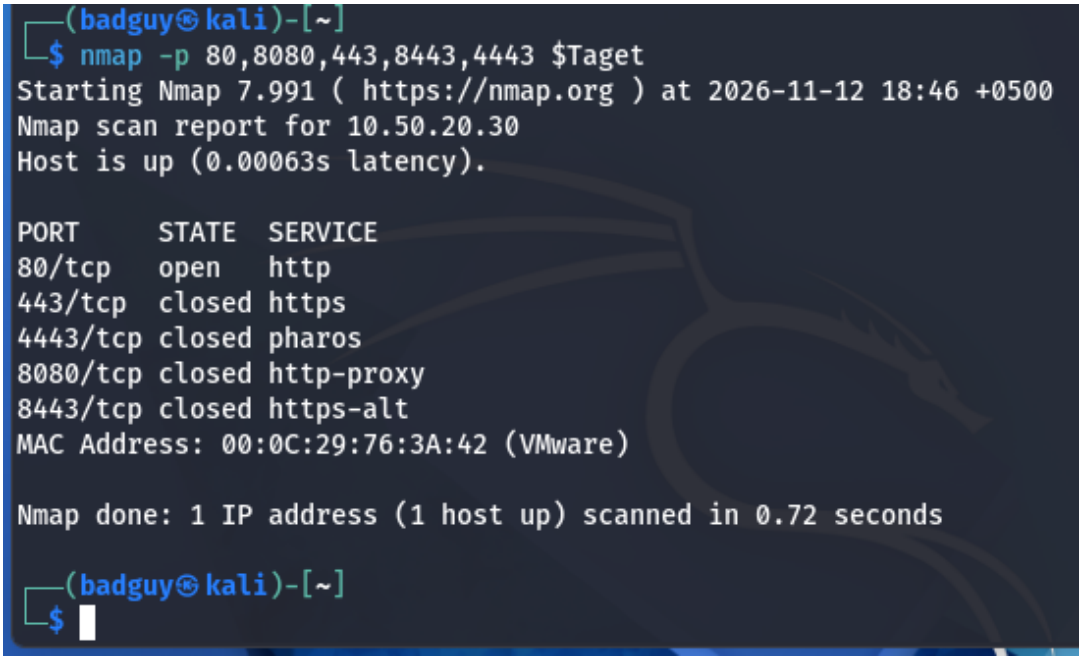


Figure 1.2: Targeted Nmap scan of web ports on 10.50.20.30 (Port 80 open).

Action Executed:	nmap -p 80,8080,8081,443,8443,4443 10.50.20.30 executed from Kali terminal.
Observation:	Port 80 (http) is open. Ports 443, 8080, 8081, 8443, 4443 are closed.
Why It Behaves This Way:	Apache daemon listens exclusively on plain HTTP port 80. No SSL/TLS listener is configured.
How to Fix (Remediation):	Enable HTTPS (port 443) with strong TLS 1.2/1.3 ciphers, enforce HSTS, and redirect port 80 to 443.

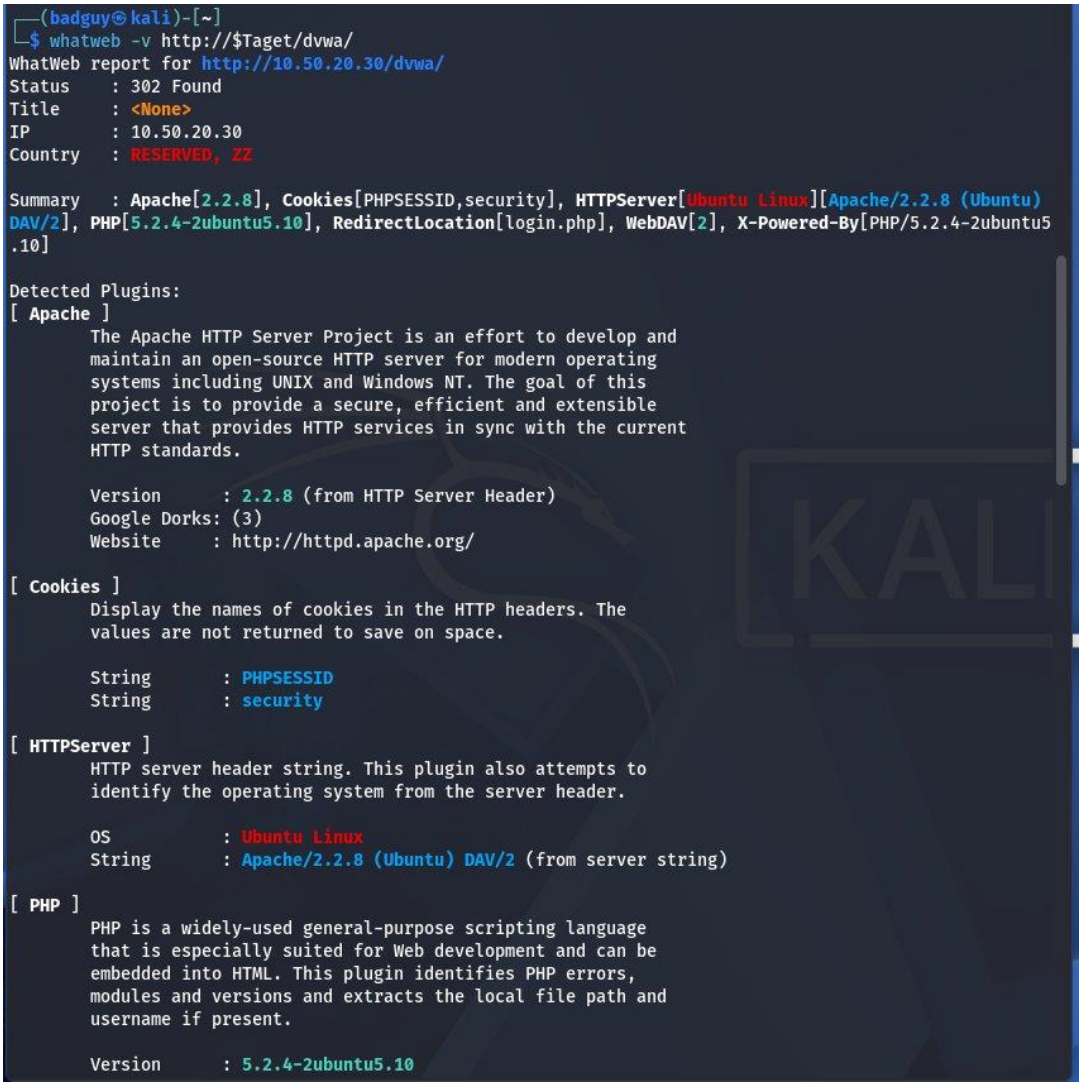


Figure 1.3: WhatWeb fingerprint identifying Apache 2.2.8 and PHP 5.2.4 on Ubuntu Linux.

Action Executed:	whatweb -v http://10.50.20.30/dvwa/ executed from Kali terminal.
Observation:	Technology fingerprint discloses Apache 2.2.8, PHP 5.2.4-2ubuntu5.10, and Ubuntu OS.
Why It Behaves This Way:	Apache and PHP broadcast version details via Server and X-Powered-By HTTP response headers.
How to Fix (Remediation):	Set ServerTokens Prod and ServerSignature Off in httpd.conf; set expose_php = Off in php.ini.

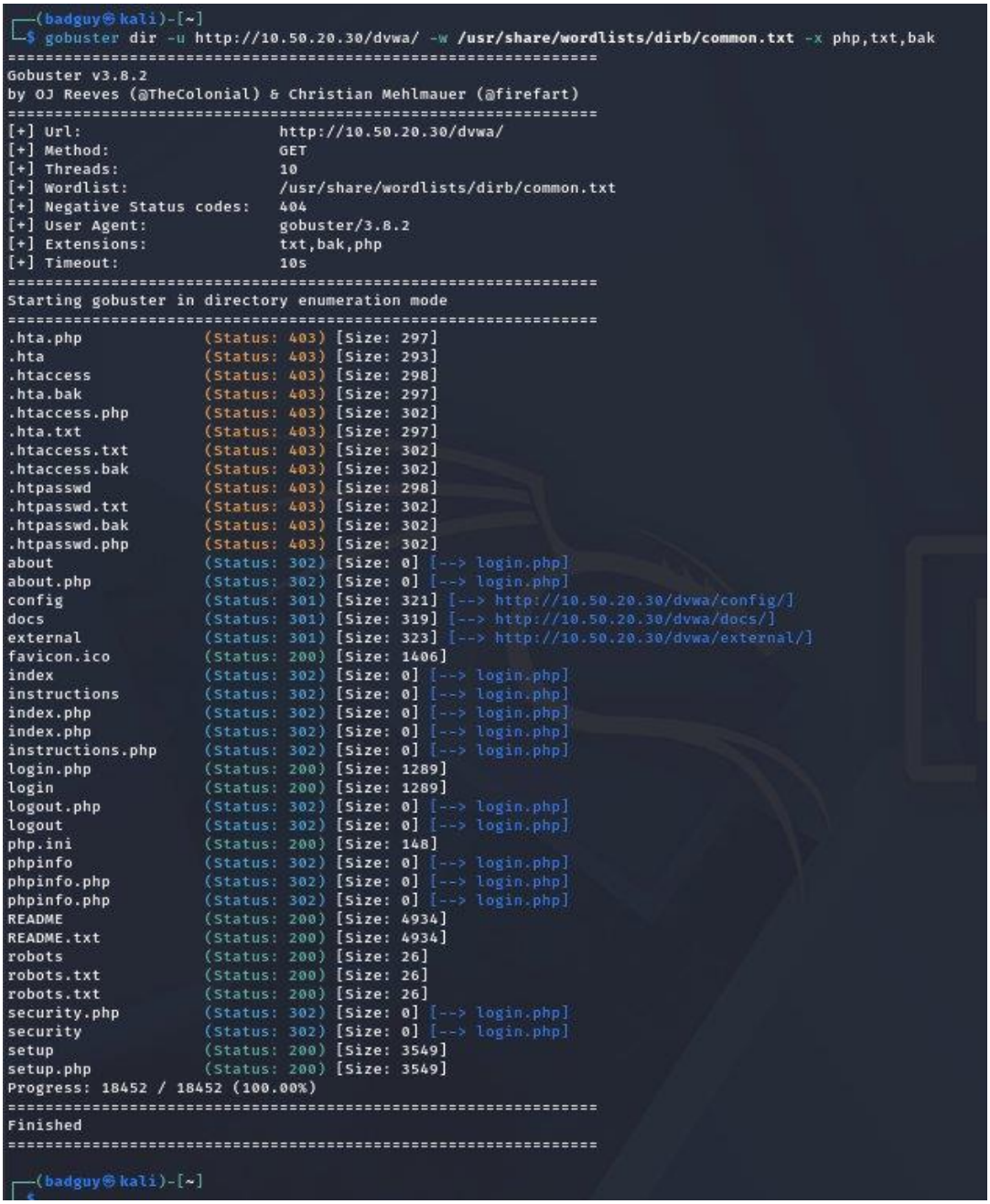


Figure 1.4: Gobuster directory brute-force discovering /hackable/uploads/.

Action Executed:	gobuster dir -u http://10.50.20.30/dvwa/ -w common.txt -x php,txt,bak executed.
Observation:	Discovered /hackable/uploads/ (301), /config/ (301), /phpinfo.php (200), /setup.php (200).
Why It Behaves This Way:	Apache serves directories directly from filesystem web root without access restriction rules.
How to Fix (Remediation):	Disable directory browsing (Options -Indexes), remove test/setup files, and restrict access via VirtualHost.

2. Level Low: Zero Server-Side Validation

Defensive Posture: At Level Low, the application performs no validation whatsoever on the uploaded file. The file is copied directly into the web-accessible directory ../../hackable/uploads/ using its client-provided filename.

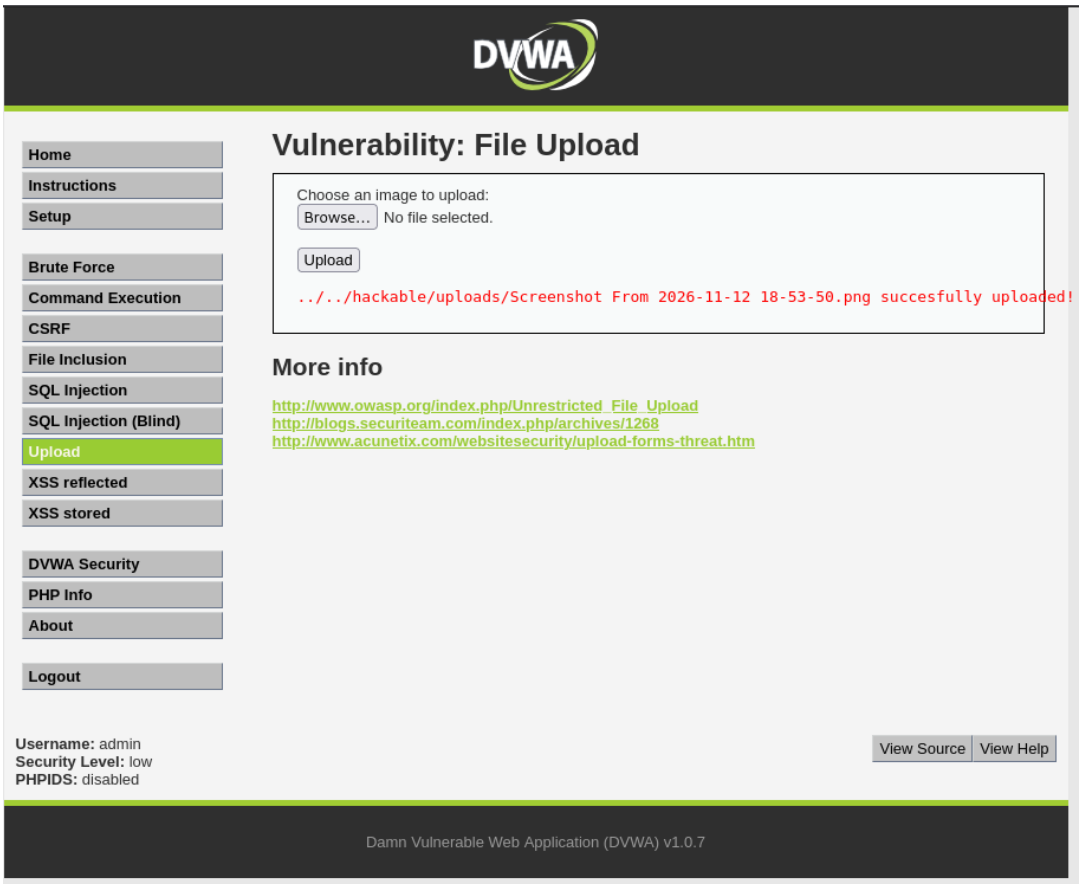


Figure 2.1: Benign PNG upload confirming storage path (../../hackable/uploads/).

Action Executed:	Uploaded legitimate PNG image through DVWA Low upload form.
Observation:	Server returned: '../../hackable/uploads/Screenshot...png succesfully uploaded!'.
Why It Behaves This Way:	Establishes baseline: upload works and server openly discloses internal destination path.
How to Fix (Remediation):	Suppress internal path disclosure in user messages; store uploads in non-web-accessible locations.



Figure 2.2: Local creation and verification of shell.php on Kali Linux.

Action Executed:	Created shell.php containing <?php if(isset($_GET['cmd'])){ system($_GET['cmd']); } ?>.
Observation:	Script confirmed created locally; designed to pass cmd parameter to

	system().
Why It Behaves This Way:	Demonstrates standard server-side scripting passing inputs to operating system functions.
How to Fix (Remediation):	Disable dangerous PHP execution functions via disable_functions = system, exec, shell_exec, passthru.

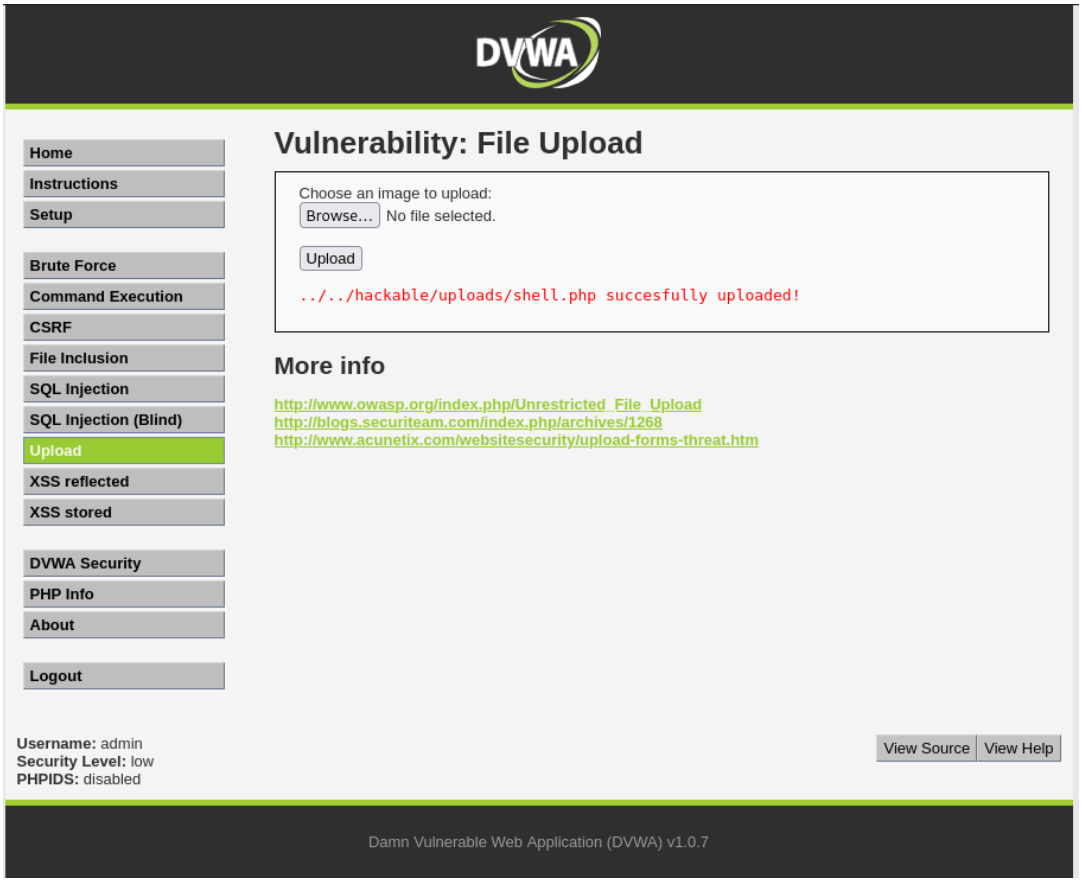


Figure 2.3: Low level confirmation: shell.php successfully stored in web root.

Action Executed:	Uploaded shell.php through DVWA Low upload module.
Observation:	Server confirmed: '../../hackable/uploads/shell.php succesfully uploaded!'.
Why It Behaves This Way:	low.php lacks extension, MIME type, or content filters; copies file directly via move_uploaded_file().
How to Fix (Remediation):	Implement strict extension allowlist, validate content, and store uploads outside web root.

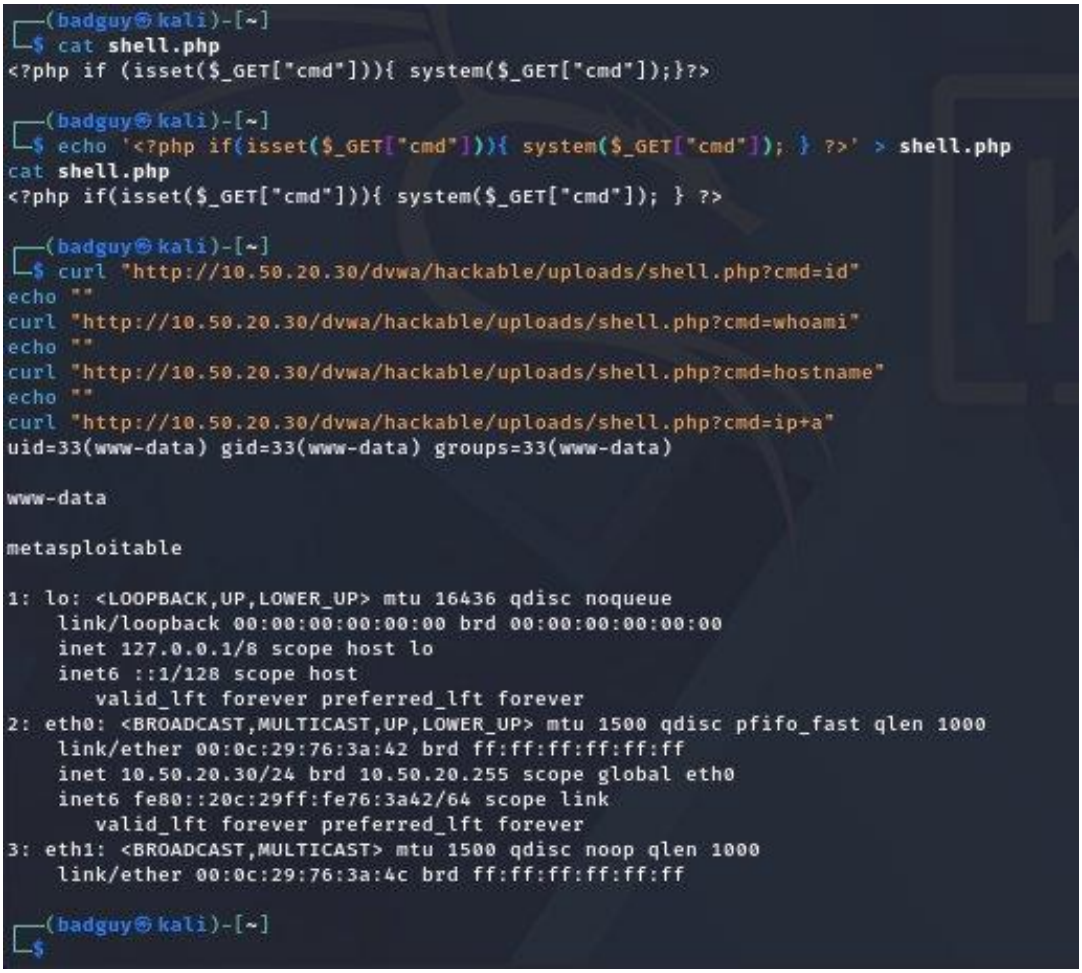


Figure 2.4: Command execution proof via curl confirming uid=33(www-data).

Action Executed:	Executed curl 'http://10.50.20.30/dvwa/hackable/uploads/shell.php?cmd=id' (and whoami, hostname, ip a).
Observation:	Output returned uid=33(www-data), hostname: metasploitable, ip: 10.50.20.30.
Why It Behaves This Way:	Apache has active PHP handler (AddType application/x-httpd-php .php) executing scripts in upload folder.
How to Fix (Remediation):	Deny script execution in upload directory via .htaccess (php_flag engine off; Options -ExecCGI).

3. Level Medium: Client-Controlled MIME Type Validation

Defensive Posture: At Level Medium, the developer attempts validation by checking that $_FILES['uploaded']['type'] matches image/jpeg or image/png and size is under 100,000 bytes. However, this value is extracted directly from the client's HTTP header.

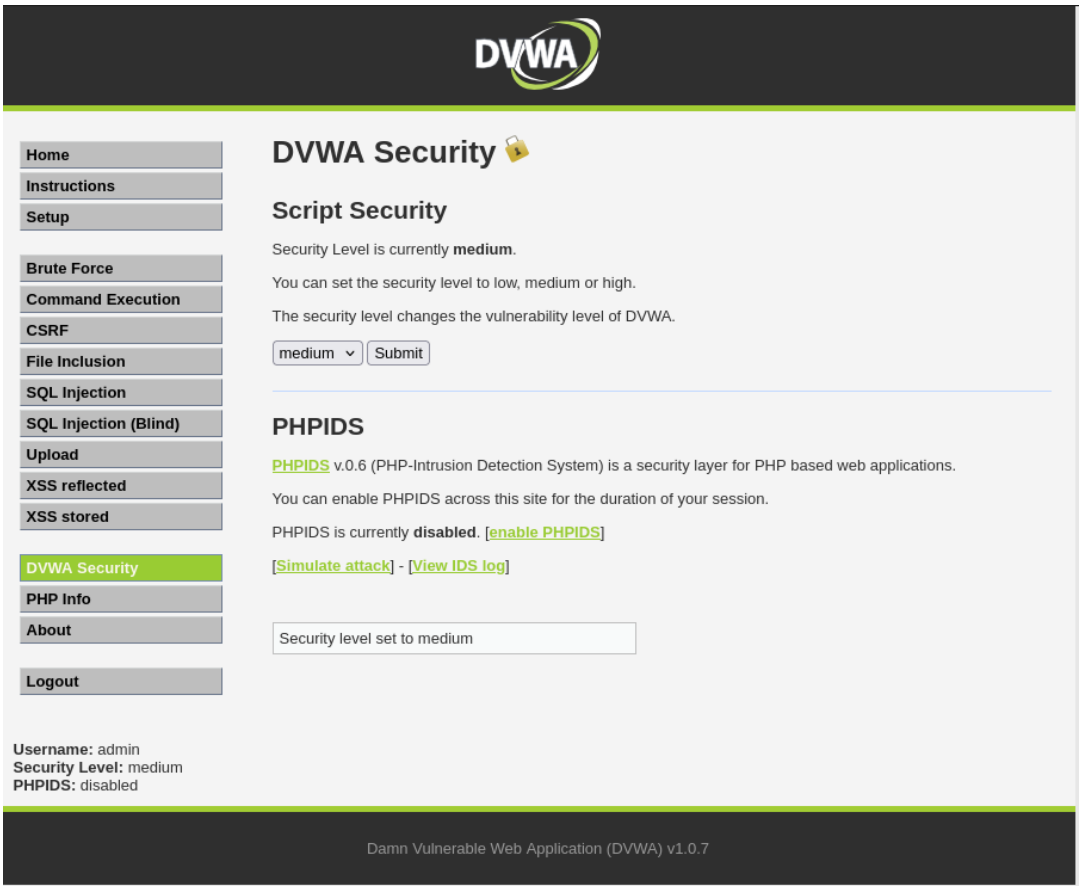


Figure 3.1: DVWA security configuration updated to Medium.

Action Executed:	DVWA Security set to Medium and submitted.
Observation:	Security level updated to 'medium'. Upload requests now route to medium.php.
Why It Behaves This Way:	Session cookie updated; activates intermediate filter logic checking $_FILES['uploaded']['type'].
How to Fix (Remediation):	Defensive controls must be enforced globally in application code rather than conditionally.

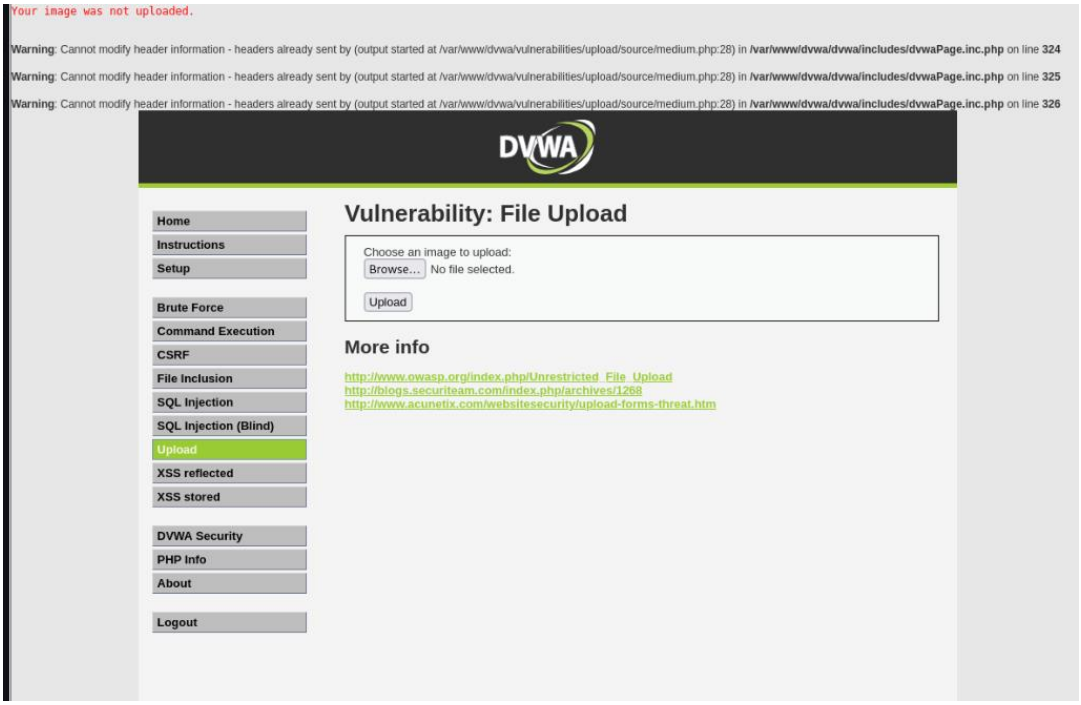


Figure 3.2: Server rejecting upload when client sends Content-Type: application/x-php.

Action Executed:	Attempted direct upload of shell.php through browser.
Observation:	Server rejected upload: 'Your image was not uploaded.'.
Why It Behaves This Way:	Browser detected .php extension and set Content-Type: application/x-php, which failed the if check.
How to Fix (Remediation):	Relying on client-supplied headers provides zero defense; validate content independently on server.

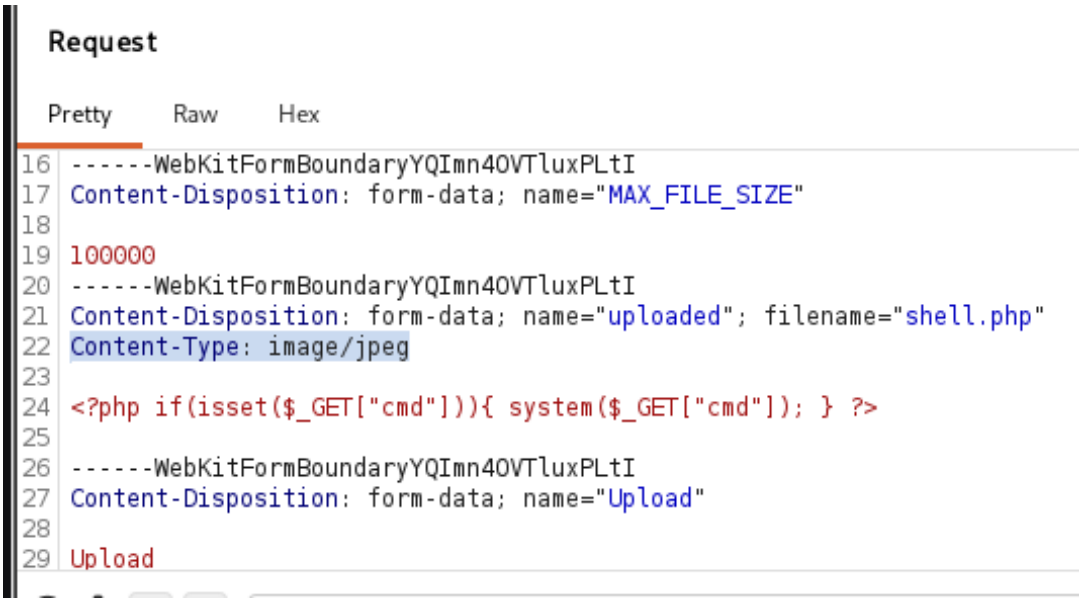


Figure 3.3: Burp Suite intercepting request and rewriting Content-Type to image/jpeg.

Action Executed:	Burp Suite intercepted POST request; Content-Type modified from application/x-php to image/jpeg.
Observation:	HTTP request held in proxy; Content-Type rewritten while

	filename='shell.php' remains.
Why It Behaves This Way:	Proxies sit between client and server; any client-supplied header can be modified before arrival.
How to Fix (Remediation):	Server must verify magic bytes using mime_content_type() instead of trusting $_FILES['uploaded']['type'].

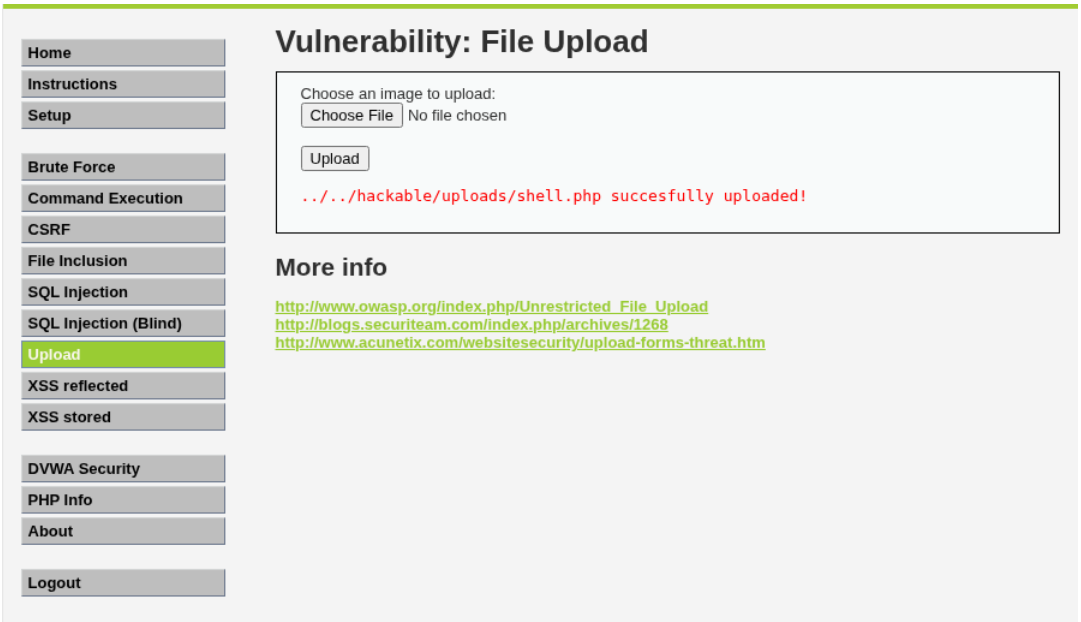


Figure 3.4: Server accepting modified request and confirming upload of shell.php.

Action Executed:	Forwarded modified request from Burp Suite to server.
Observation:	Server accepted upload: '../../hackable/uploads/shell.php succesfully uploaded!'.
Why It Behaves This Way:	medium.php evaluated falsified Content-Type: image/jpeg header, passed check, and saved script.
How to Fix (Remediation):	Enforce strict extension allowlist, server-side content parsing, and execution denial.

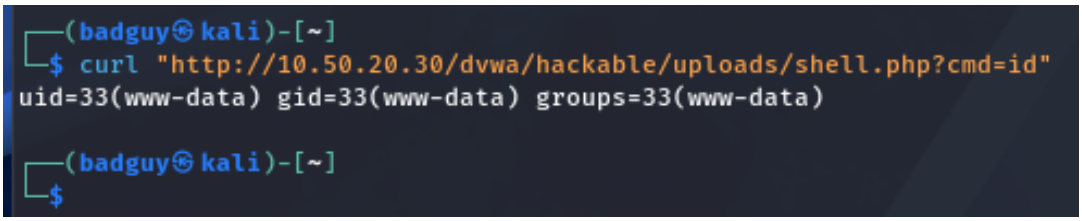


Figure 3.5: Remote command execution verified via curl on Medium security.

Action Executed:	Executed curl 'http://10.50.20.30/dvwa/hackable/uploads/shell.php?cmd=id'.
Observation:	Returned uid=33(www-data) gid=33(www-data) groups=33(www-data).
Why It Behaves This Way:	File was saved with .php extension in web root; Apache executed it identically to Low level.
How to Fix (Remediation):	Disable PHP execution handlers in uploads directory and randomize stored filenames.

4. Level High: Extension Whitelist & Content Inspection

Defensive Posture: Level High implements strict extension checking (.jpg, .jpeg, .png) and runs PHP's getimagesize() function on the file. Direct scripts are rejected, but dual-format polyglots defeat content inspection.

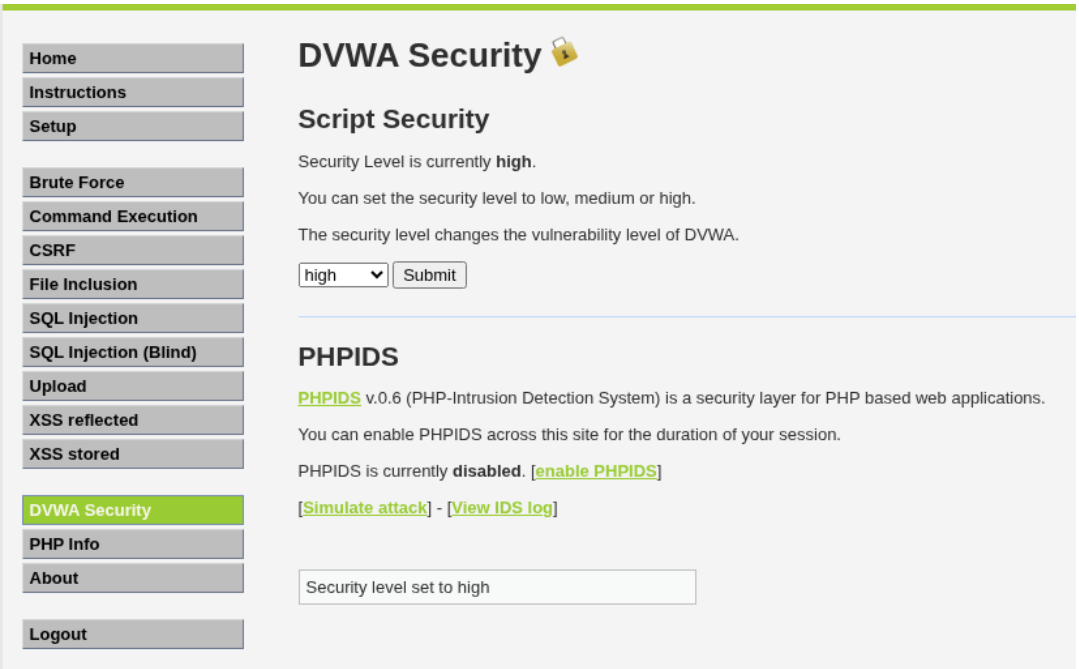


Figure 4.1: DVWA security configuration updated to High.

Action Executed:	DVWA Security set to High and submitted.
Observation:	Security level updated to 'high'. Upload requests now route to high.php.
Why It Behaves This Way:	Session cookie updated; activates strict extension allowlist and getimagesize() validation.
How to Fix (Remediation):	Security architectures must integrate defense-in-depth across storage and runtime.

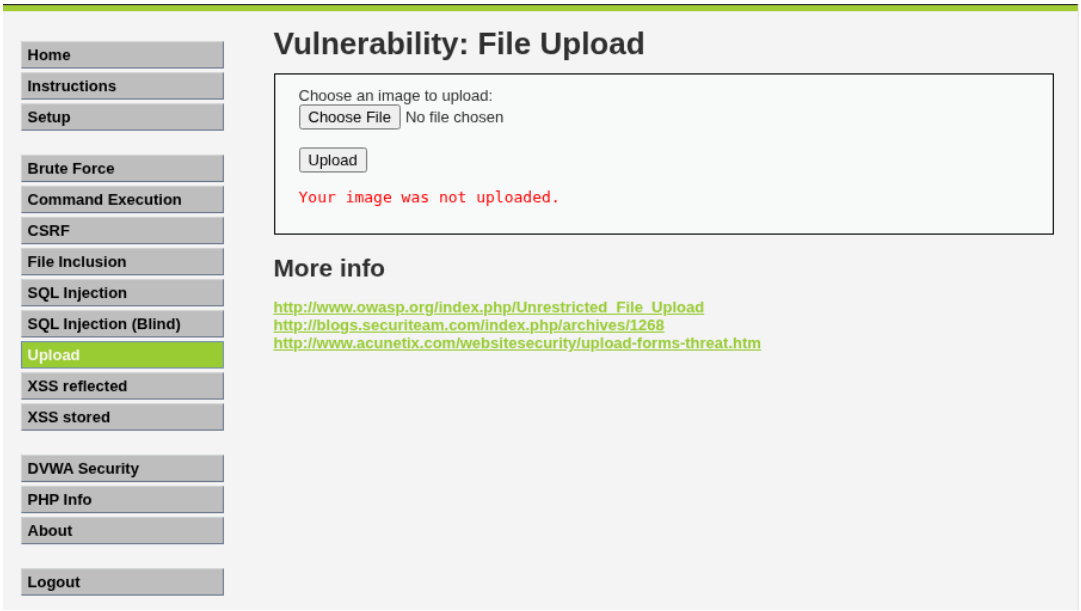


Figure 4.2: High security rejecting script upload despite MIME manipulation.

Action Executed:	Attempted upload of shell.php with Burp modifying Content-Type to image/jpeg.
Observation:	Server rejected upload: 'Your image was not uploaded.'.
Why It Behaves This Way:	high.php inspects extension (.php fails allowlist) and getimagesize() fails on plain text.
How to Fix (Remediation):	Extension checks and dimension checks are necessary, but insufficient without re-encoding.



Figure 4.3: Terminal output showing creation and validation of dual-format JPEG polyglot.

Action Executed:	Created 50x50 JPEG with convert, appended PHP code with printf, verified with file and identify.
Observation:	file and identify confirm file remains valid JPEG image (50x50, JFIF 1.01).
Why It Behaves This Way:	PHP code sits after JPEG End-of-Image marker (0xFF 0xD9); image tools read header; PHP reads tags.
How to Fix (Remediation):	Re-encode all uploaded images using GD (imagecreatefromjpeg/imagejpeg) to strip appended data.

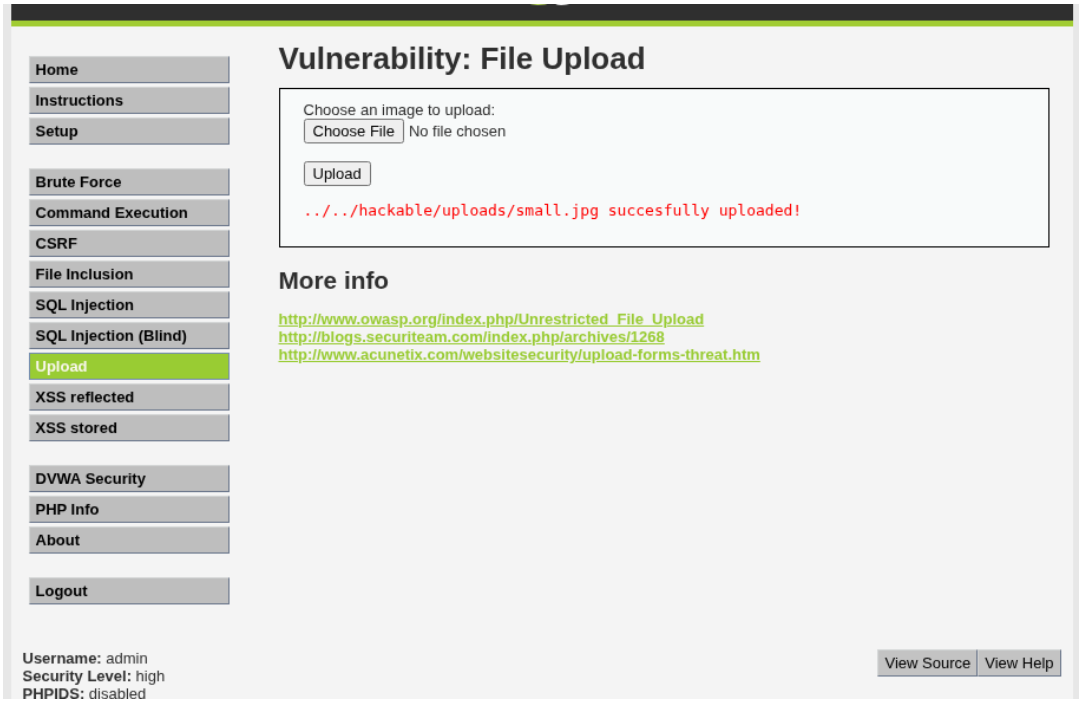


Figure 4.4: High-level filter bypassed: small.jpg accepted by getimagesize() check.

Action Executed:	Uploaded small.jpg through DVWA High upload form.
Observation:	Server accepted upload: '../../hackable/uploads/small.jpg succesfully uploaded!'.
Why It Behaves This Way:	small.jpg satisfied both checks: extension was .jpg and getimagesize() parsed valid dimensions.
How to Fix (Remediation):	In addition to extension checks, web servers must deny execution handlers in upload folders.

5. Level Impossible: Architectural Defense-in-Depth

Defensive Architecture: The Impossible level implements four independent, layered architectural controls that defeat all upload bypasses:

• **Control 1:** 1. Anti-CSRF Token Validation (checkToken): Verifies session-bound cryptographic token, blocking automated attacks.

• **Control 2:** 2. Cryptographic Filename Randomization: Original name is discarded; file saved as md5(uniqid() . $uploaded_name) . '.' . $uploaded_ext.

• **Control 3:** 3. Image GD Re-Encoding (imagecreatefromjpeg / imagejpeg): Server re-renders raw pixels into a clean image, permanently stripping appended code and metadata.

• **Control 4:** 4. Directory Execution Denial: Web server configuration (.htaccess with php_flag engine off) prevents Apache from executing scripts in the upload folder.

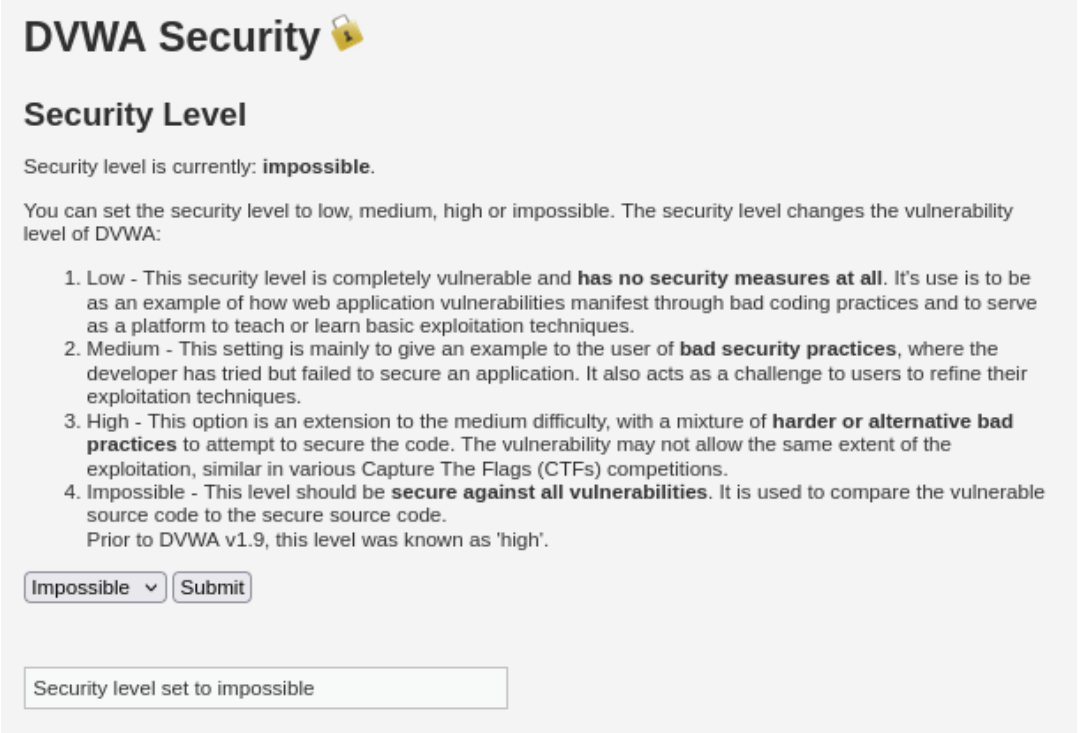


Figure 5.1: DVWA security configuration updated to Impossible.

Action Executed:	DVWA Security set to Impossible and submitted.
Observation:	Security level updated to 'impossible'. Upload requests now route to impossible.php.
Why It Behaves This Way:	Session cookie updated; activates complete defense-in-depth reference implementation.
How to Fix (Remediation):	This represents the target secure architecture to be enforced across all production applications.

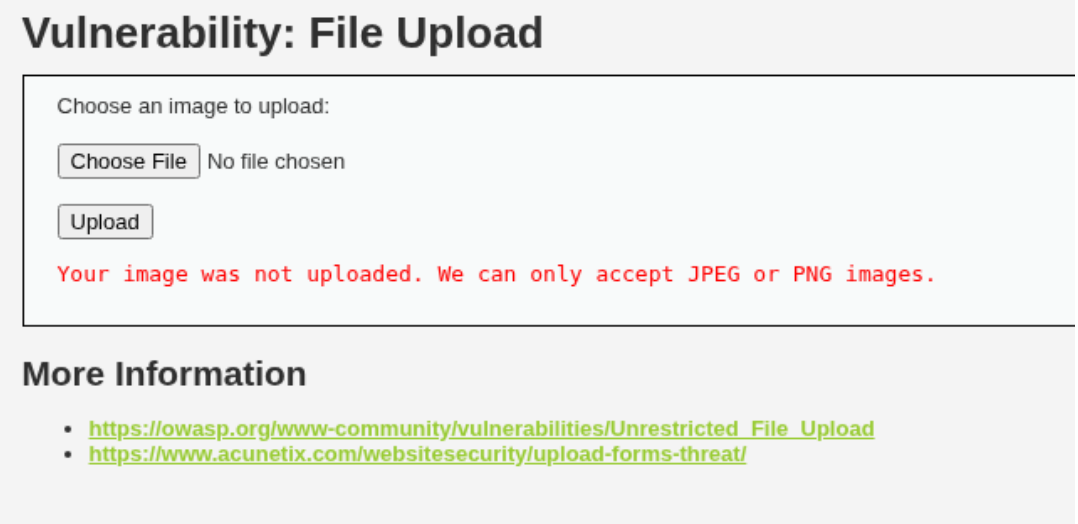


Figure 5.2: Server rejecting invalid file types with strict extension and MIME enforcement.

Action Executed:	Attempted upload of unvalidated script file through Impossible form.
Observation:	Server rejected upload: 'Your image was not uploaded. We can only accept JPEG or PNG images.'.
Why It Behaves This Way:	Multi-condition pre-validation strictly validates extension, size, MIME type, and dimensions.
How to Fix (Remediation):	This represents proper input validation filtering.

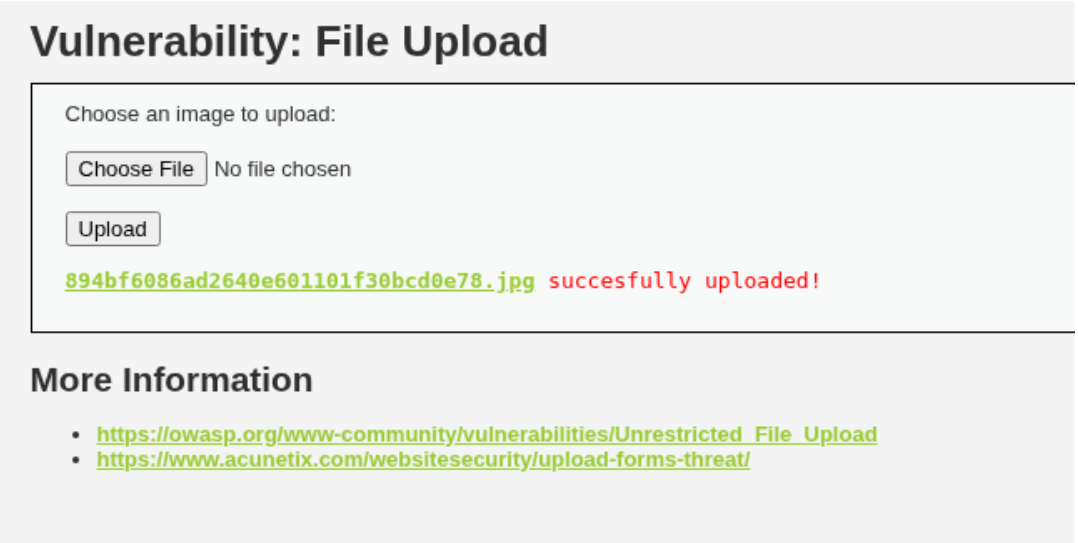


Figure 5.3: Server accepting image under a randomized MD5 filename after pixel re-encoding.

Action Executed:	Uploaded small.jpg through Impossible form.
Observation:	Server stored file under randomized MD5 name: '894bf6086ad2640e601101f30bcd0e78.jpg succesfully uploaded!'.
Why It Behaves This Way:	GD library re-rendered raw pixels and discarded user filename; appended code was permanently stripped.
How to Fix (Remediation):	This is the recommended defensive pattern: cryptographic naming + pixel re-encoding.

```php
<?php

if( isset( $_POST[ 'Upload' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );


    // File information
    $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
    $uploaded_ext  = substr( $uploaded_name, strrpos( $uploaded_name, '.' ) + 1);
    $uploaded_size = $_FILES[ 'uploaded' ][ 'size' ];
    $uploaded_type = $_FILES[ 'uploaded' ][ 'type' ];
    $uploaded_tmp  = $_FILES[ 'uploaded' ][ 'tmp_name' ];

    // Where are we going to be writing to?
    $target_path   = DVWA_WEB_PAGE_TO_ROOT . 'hackable/uploads/';
    //$target_file   = basename( $uploaded_name, '.' . $uploaded_ext ) . '-';
    $target_file   =  md5( uniqid() . $uploaded_name ) . '.' . $uploaded_ext;
    $temp_file     = ( ( ini_get( 'upload_tmp_dir' ) == '' ) ? ( sys_get_temp_dir() ) : ( ini_get( 'upload_tmp_dir' ) ) );
    $temp_file    .= DIRECTORY_SEPARATOR . md5( uniqid() . $uploaded_name ) . '.' . $uploaded_ext;

    // Is it an image?
    if( ( strtolower( $uploaded_ext ) == 'jpg' || strtolower( $uploaded_ext ) == 'jpeg' || strtolower( $uploaded_ext ) == 'png' ) &&
        ( $uploaded_size < 100000 ) &&
        ( $uploaded_type == 'image/jpeg' || $uploaded_type == 'image/png' ) &&
        getimagesize( $uploaded_tmp ) ) {

        // Strip any metadata, by re-encoding image (Note, using php-Imagick is recommended over php-GD)
        if( $uploaded_type == 'image/jpeg' ) {
            $img = imagecreatefromjpeg( $uploaded_tmp );
            imagejpeg( $img, $temp_file, 100);
        }
        else {
            $img = imagecreatefrompng( $uploaded_tmp );
            imagepng( $img, $temp_file, 9);
        }
        imagedestroy( $img );

        // Can we move the file to the web root from the temp folder?
        if( rename( $temp_file, ( getcwd() . DIRECTORY_SEPARATOR . $target_path . $target_file ) ) ) {
            // Yes!
            echo "<pre><a href='{$target_path}{$target_file}'>{$target_file}</a> succesfully uploaded!</pre>";
        }
        else {
            // No
            echo '<pre>Your image was not uploaded.</pre>';
        }

        // Delete any temp files
        if( file_exists( $temp_file ) )
            unlink( $temp_file );
    }
    else {
        // Invalid file
        echo '<pre>Your image was not uploaded. We can only accept JPEG or PNG images.</pre>';
    }
}

// Generate Anti-CSRF token
generateSessionToken();

?>
```

Figure 5.4: DVWA View Source window displaying the secure reference implementation in impossible.php.

Action Executed:	Clicked 'View Source' button in bottom-right corner of DVWA Upload module.
Observation:	Opened modal window displaying full PHP source code of impossible.php.
Why It Behaves This Way:	DVWA provides reference implementation source code to educate developers on secure coding.
How to Fix (Remediation):	Use this reference implementation as the baseline engineering standard for all upload endpoints.

Task 3: Executive Remediation Brief (Weight: 25%)

TO:	Executive Leadership & IT Steering Committee
FROM:	Enterprise Information Security Assessment Team
DATE:	November 12, 2026
SUBJECT:	Strategic Risk Assessment & Remediation Roadmap: File Storage Architecture

Executive Risk Analysis & Business Exposure

During our recent technical security assessment, our team evaluated the mechanisms responsible for receiving external customer files within our core web applications. We identified a critical architectural weakness in how incoming files are stored and managed on corporate infrastructure.

Under current configurations, our servers accept files submitted by internet users and store them directly inside public website directories. Crucially, organizational web servers are configured to treat files in these directories as executable application code rather than inert data. This creates four major areas of commercial and operational exposure:

- 1. Total Loss of Data Confidentiality: Unauthorized external actors can read proprietary application databases, customer records, employee credentials, and backend configuration files.
- 2. Complete Host Compromise: External parties can execute administrative commands on organizational servers, establishing an unauthorized foothold inside our internal network.
- 3. Critical Business Disruption: External actors can alter website contents, delete core databases, or deploy encrypting software that halts corporate operations.
- 4. Regulatory Penalties: An exposure of customer databases triggers mandatory data breach disclosures under privacy legislation, carrying substantial financial fines and lasting brand harm.

Root Cause in Plain Language

The risk does not arise simply from allowing users to upload documents or profile pictures. The danger arises from a fundamental failure to separate storage from execution:

- 1. Excessive Trust in External Labels: Our software currently trusts declarations provided by the user's internet browser (such as file names and claimed format types) without independently verifying what the file actually contains.
- 2. Execution Permissions on Storage Folders: When an external file is placed in a public folder, the server's default configuration attempts to run any code it recognizes inside that file. The server should never run software out of a folder dedicated to storing images or documents.

Four-Pillar Strategic Remediation Roadmap

To permanently eliminate this vulnerability across all organizational systems, engineering teams must implement four mandatory controls:

Pillar 1 — 1. Server-Side Image Re-generation: Applications must never store raw files received from external users. Graphics must be processed through backend rendering software that reads raw pixels and writes a brand new, clean image file. This process automatically strips all unauthorized instructions, comments, and hidden data.

Pillar 2 — 2. Cryptographic Name Replacement: Discard original user-provided filenames entirely. The system must assign a randomized, computer-generated alphanumeric identifier with an approved extension, preventing attackers from predicting file locations or manipulating directory paths.

Pillar 3 — 3. Storage Outside Public Web Roots: Store uploaded documents in private storage locations—such as dedicated cloud object storage buckets or isolated internal directories—that cannot be accessed directly via internet web browsers.

Pillar 4 — 4. Absolute Execution Denial: Web server software must be explicitly configured so that program execution handlers are permanently deactivated across all file storage repositories. The server must be fundamentally incapable of running stored files as software.

Executive Recommendation

Remediating this critical weakness does not require expensive commercial licenses. It requires enforcing disciplined engineering policies that isolate data storage from program execution. We recommend mandating these four pillars across all development teams immediately.